Laboratorio 9

Introducción - El problema del cuadrado mágico.

El problema del cuadrado mágico consiste en colocar los números del 1 al n² en una cuadrícula de n × n (sin repetir números) de tal forma que la suma de los números de cada fila, cada columna y las dos diagonales (principal y secundaria) sea siempre la misma, conocida como la constante mágica (S).

$$S = \frac{n(n^2+1)}{2}$$

Método de resolución

• Backtracking (Vuelta atrás): Es una de las técnicas para resolverlo. Consiste en colocar los dígitos de 1 a n^2, en cada una de las celdas. No deben existir dígitos repetidos en las celdas de la cuadrícula. Una vez completada la tarea (no hay celdas desocupadas), se procede a verificar si tenemos un cuadrado mágico.

1. Para saberlo se procede a calcular la suma de cada fila y compararlas entre sí.
2. Si no coinciden, entonces no es cuadrado mágico.
3. Si coinciden, entonces ahora debe verificarse que este valor también se puede obtener al sumar los valores de las celdas en cada columna.
4. Si no coinciden, entonces no es cuadrado mágico
5. Si coinciden, entonces ahora debe verificarse para la suma de los valores de las celdas de la diagonal principal (esquina superior izquierda a esquina inferior derecha) y de la diagonal secundaria (esquina superior derecha a esquina inferior izquierda).
6. Si no coinciden, entonces no es cuadrado mágico.
7. Si coinciden, entonces se ha probado que es un cuadrado mágico.

Tablero de 3x3

8	3	4
1	5	9
6	7	2

Definición del problema y requisitos de la solución

El objetivo es implementar en Python un algoritmo que permita encontrar solución (si es que la hay) , para un tablero de NxN.
Se pide:

1. Implementar el algoritmo que permita saber si existe o no solución cuando N = 3,4, 5, 6, 7, 8. Debe guardar las soluciones que encuentre para cada N, en un archivo (solucion.txt). (50 % nota)
2. Implementar el algoritmo que permita encontrar todas las soluciones (si existen) para cada N(Todas_las_soluciones.txt). (30 % nota)
3. Implementar el algoritmo que permita encontrar la mejor solución (si existe) al problema (cuadrado_mágico.txt). (20 % nota).

Plantilla Backtracking iterativo - una solución

```
begin
        #definición particular de cada problema
        while(tenga candidatos AND not solución)do
                if(valido movimiento elegido)then
                        avanzar en la búsqueda de la solución
                        if(llegó al final) then
                                solucion = true
                        else
                                dejar pistas para devolverme
                        endif
                else
                        siguiente candidato
                        while(no estoy en el inicio AND ya no tengo candidatos)do
                                retrocedo
                        endwhile
                endif
        endwhile
end
```

Instrucciones

1. El laboratorio deberá ser desarrollado de manera **individual** utilizando Python.

2. La solución deberá implementarse en un archivo con extensión **.py**, el cual deberá contener el algoritmo solicitado de acuerdo con los requisitos definidos en el laboratorio.
3. El programa deberá ejecutarse correctamente, sin presentar errores de compilación o ejecución, y cumplir con todas las funcionalidades especificadas para el algoritmo.
4. El código deberá seguir los principios de la programación estructurada, utilizando adecuadamente variables, funciones, estructuras de control y demás elementos vistos durante el curso.
5. Se deberán considerar los casos de prueba necesarios para verificar el correcto funcionamiento del programa, incluyendo la validación de las entradas cuando corresponda.
6. Utilizar como base de trabajo Pseudo código en el ANEXO

Evaluación.

1. La entrega del laboratorio será **individual** y deberá realizarse dentro del plazo establecido por el docente.
2. La evaluación se regirá por la **Pauta de Evaluación de Laboratorio** correspondiente a la asignatura, considerando tanto el correcto funcionamiento del programa como el cumplimiento de los requisitos establecidos.
3. Como parte de la evaluación, el estudiante deberá realizar una **defensa del código desarrollado**, en la cual deberá explicar la lógica de la solución implementada, el funcionamiento del algoritmo y las decisiones tomadas durante su desarrollo.
4. Durante la defensa, el docente podrá realizar preguntas relacionadas con el código presentado y solicitar modificaciones o explicaciones sobre partes específicas del programa, con el fin de verificar la comprensión del estudiante y la autoría del trabajo entregado.
5. Todos los trabajos serán evaluados en el laboratorio
6. **FECHA DE ENTREGA: 18 DE OCTUBRE DE 2026 HASTA LAS 24 HORAS.**

ANEXO

```
#################################################################
#     BEGIN MODULO PRINCIPAL CUADRADO MAGICO
#################################################################
X,Y = 0, 0

CANDIDATO = 1; SOLUCION = FALSE ; X_INICIAL = 0 ;

Y_INICIAL = 0 WHILE CANDIDATO <= N*N AND SOLUCION

== FALSE DO

        IF CANDIDATO_ES_VALIDO(TABLERO,

            CANDIDATO)THEN TABLERO[X][Y] =

            CANDIDATO

        IF LLEGO_AL_FINAL(TABLERO, N)THEN

                IF CUADRADO_MAGICO(TABLERO, N)

                    THEN SOLUCION = TRUE

            ELSE

                CANDIDATO = CANDIDATO + 1 ; TABLERO[X][Y] = 0

                    WHILE (X != X_INICIAL AND Y != Y_INICIAL ) AND

                        (CANDIDATO == 10) DO X, Y =

                        DEVOLVER(TABLERO, X, Y)

                    CANDIDATO = TABLERO[X][Y] + 1

                    TABLERO[X][Y] = 0

                ENDWHILE

            ENDIF

        ELSE

            X, Y = SIGUIENTE_XY(TABLERO)

            CANDIDATO = 1

        ENDIF

    ELSE
```

```
            CANDIDATO = CANDIDATO + 1

                WHILE (X != X_INICIAL AND Y != Y_INICIAL ) AND (CANDIDATO == 10) DO
                    X, Y = DEVOLVER(TABLERO, X, Y)

                CANDIDATO = TABLERO[X][Y] + 1

                TABLERO[X][Y] = 0

            ENDWHILE

        ENDIF

ENDWHILE

#################################################################
#     BEGIN MODULO CANDIDATO_ES_VALIDO
#################################################################
BEGIN CANDIDATO_ES_VALIDO(TABLERO, CANDIDATO)

        FOR I = 0 ; I < N ; I ++

            FOR J = 0 ; J < N ; J++

                IF TABLERO[I][J] == CANDIDATO THEN

                    RETURN 0

                ENDIF

            ENDFOR

        ENDFOR

        RETURN 1

END
#################################################################
#     BEGIN LLEGO_AL_FINAL
#################################################################
BEGIN LLEGO_AL_FINAL(TABLERO, N)

        FOR I = 0 ; I < N ; I ++

            FOR J = 0 ; J < N ; J++

                IF TABLERO[I][J] == 0

                    RETURN 0

                ENDIF

            ENDFOR
```

```
        ENDFOR
        RETURN 1
END
#################################################################
#     BEGIN SIGUIENTE_XY
#################################################################
BEGIN SIGUIENTE_XY(TABLERO)
        FOR I = 0 ; I < N ; I ++
                FOR J = 0 ; J < N ; J++
                        IF TABLERO[I][J] == 0
                                RETURN (I, J)
                        ENDIF
                ENDFOR
        ENDFOR
END
#################################################################
#     BEGIN DEVOLVER
#################################################################
BEGIN DEVOLVER(TABLERO, I, J)
        J = J – 1
        IF J == -1 THEN
                I = I – 1

                J = N – 1
        ENDIF
        RETURN (I, J)
END
#################################################################
#     BEGIN CUADRADO MAGICO
#################################################################
        BEGIN
                CUADRADO_MAGICO(
                TABLERO,N) AUX =
```

```
            SUMA_FILA(TABLERO,
            0)
        I = 0
        WHILE (I<N)DO
            IF SUMA_FILA(TABLERO,I) != AUX THEN
                RETURN 0
            ENDIF
            IF SUMA_COLUMNA(TABLERO, I) != AUX THEN
                RETURN 0
            ENDIF
            I = I + 1
        ENDWHILE
        IF AUX != SUMA_DP(TABLERO) THEN
            RETURN 0
        ENDIF
        IF AUX != SUMA_DS(TABLERO) THEN
            RETURN 0
        ENDIF
        RETURN 1 # ES CUADRADO MAGICO
END
#################################################################
#     BEGIN SUMA_FILA
#################################################################
BEGIN SUMA_FILA(TABLERO, FILA)
        SUMA = 0
        FOR J = 0 ; J=N ; J++
            SUMA = SUMA + TABLERO[FILA][J]
        ENDFOR
```

```
        RETURN SUMA
END
#################################################################
#     BEGIN SUMA_COLUMNA
#################################################################
BEGIN SUMA_COLUMNA(TABLERO, COLUMNA)
        SUMA = 0
        FOR J = 0 ; J=N ; J++
                SUMA = SUMA + TABLERO[J][COLUMNA]
        ENDFOR
        RETURN SUMA
END
#################################################################
#     BEGIN SUMA_DP
#################################################################
BEGIN SUMA_DP(TABLERO)
        SUMA = 0
        FOR J = 0 ; J=N ; J++
                SUMA = SUMA + TABLERO[J][J]
        ENDFOR
        RETURN SUMA
END
#################################################################
#     BEGIN SUMA_DS
#################################################################
BEGIN SUMA_DS(TABLERO)
        SUMA = 0 ; J = N - 1
        FOR I = 0 ; I<N ; I++
                SUMA = SUMA + TABLERO[I][J] J = J - 1
        ENDFOR RETURN SUMA
END
```